Lesson plan · for the teacher

When the model is wrong

Optimization · Level 4 · Lesson 3

Time
40 min

Materials
Board and chalk; The cheapest-lunch plan written on a card or the board; A blank “assumptions” sheet per group

Learning goals

- Find the rules a model is missing by testing its answer with people.
- Check whether a plan jumps when a shaky number changes.
- Write a model’s assumptions down so others can check them.

Activity: Break the model

1. Warm-up (5 min): Write the computer’s cheapest lunch on the board: 1.3 cups of rice, 2 handfuls of groundnuts, half a cup of greens, every day. Ask: would you eat this every day? Why not?
2. Teach (7 min): Explain that the model only knew prices and three needs (energy, protein, iron), so it did exactly what it was told. Its sums are right; its rules are incomplete.
3. Break it (12 min): Groups play the cooks, pupils and parents. Each group lists every objection it can find: taste, variety, allergies, cooking time, fractions of a cup, religious or family food rules.
4. Fix it (10 min): Groups sort their objections into missing rules and wrong numbers, and write each rule as a sentence a model could check, such as “at most 1 handful of groundnuts”.
5. Assumptions (6 min): Each group writes three lines: what the model meets, what data it used, and what it still ignores. Pin them up beside the plan.

No lunch at school? Use the timetable instead: show a timetable with no clashes but maths always in the last period, and ask pupils to break it.

Discussion

- Who in our school would notice a missing rule first, and why?
- Which numbers in our plans are we least sure of?
- Why might a perfect solver still give a bad plan?

Exit ticket

- Write one missing rule for the cheapest lunch, as a sentence a model could check.

Try it at school tomorrow
Show pupils a plan a computer could make, such as the cheapest lunch (rice, two handfuls of groundnuts and a little greens, every day), and ask them to break it. Write every objection on the board as a new rule, then ask: which rules did the first model miss, and who would have noticed?

When the model is wrong

1 **The lunch model's cheapest plan looks fine to the pupils, but the cooks say it can't be cooked in time for the lunch break. Who is right about the model?**

◯ The pupils: they eat it ◯ The cooks: cooking time is a missing limit ◯ Nobody: the model is optimal

2 **This lunch model says lunch costs 36.77 coins, but the plate gives only 1.75 mg of iron instead of 5. One line has the mistake. Which one?**

Circle the line with the mistake. Write it correctly:

```
1  from ortools.linear_solver import pywraplp
2  s = pywraplp.Solver.CreateSolver("GLOP")
3  rice, beans, nuts, greens = [s.NumVar(0, 2, f)
4      for f in ("rice", "beans", "nuts", "greens")]
5  s.Add(260*rice + 230*beans + 170*nuts + 40*greens >= 700)
6  s.Add(5*rice + 15*beans + 7*nuts + 5*greens >= 20)
7  # s.Add(0.4*rice + 4*beans + 0.6*nuts + 6*greens >= 5)
8  s.Minimize(15*rice + 20*beans + 8*nuts + 10*greens)
9  s.Solve()
```

3 **You show the cheapest lunch to cooks, pupils and parents. Which of their objections become new limits in the model? Choose all that apply.**

☐ "The printout is hard to read" ☐ "My son is allergic: no groundnuts on his plate"
☐ "No more than one handful of groundnuts" ☐ "Rice now costs 18 coins, not 15"
☐ "We can only cook 1 cup of rice per pupil before 12:30"

4 **The cooks' rule: no more than one handful of groundnuts per pupil. Add it to the lunch model and run it. What does lunch cost now?**

```
from ortools.linear_solver import pywraplp
s = pywraplp.Solver.CreateSolver("GLOP")
rice = s.NumVar(0, 2, "rice")
beans = s.NumVar(0, 2, "beans")
nuts = s.NumVar(0, 2, "nuts")
greens = s.NumVar(0, 2, "greens")
s.Add(260*rice + 230*beans
      + 170*nuts + 40*greens >= 700)
s.Add(5*rice + 15*beans
      + 7*nuts + 5*greens >= 20)
s.Add(0.4*rice + 4*beans
      + 0.6*nuts + 6*greens >= 5)
s.Minimize(15*rice + 20*beans
           + 8*nuts + 10*greens)
s.Solve()
print(round(s.Objective().Value(), 2))
```

Write your program:

5 **A plan the solver reports as OPTIMAL still needs testing with the people who will live with it.**

◯ True ◯ False

✎ **Think of a plan your school uses that was made by a rule or a computer: a timetable, a rota, a menu. Which rule might it be missing, and who would notice first?**

Check your answers

When the model is wrong

1 **The cooks: cooking time is a missing limit**

A plan that can't be cooked in time is not feasible in the real kitchen, however good it looks on paper.

2 **Line 7: `s.Add(0.4*rice + 4*beans + 0.6*nuts + 6*greens >= 5)`**

The iron limit was commented out, perhaps while someone was testing, so the solver never saw it and happily dropped the greens.

3 **"No more than one handful of groundnuts"; "My son is allergic: no groundnuts on his plate"; "We can only cook 1 cup of rice per pupil before 12:30"**

The groundnut limit, the allergy and the cooking capacity are rules the plan must keep, so they become limits.

4

```
from ortools.linear_solver import pywraplp
s = pywraplp.Solver.CreateSolver("GLOP")
rice = s.NumVar(0, 2, "rice")
beans = s.NumVar(0, 2, "beans")
nuts = s.NumVar(0, 2, "nuts")
greens = s.NumVar(0, 2, "greens")
s.Add(260*rice + 230*beans
      + 170*nuts + 40*greens >= 700)
s.Add(5*rice + 15*beans
      + 7*nuts + 5*greens >= 20)
s.Add(0.4*rice + 4*beans
      + 0.6*nuts + 6*greens >= 5)
s.Add(nuts <= 1)
s.Minimize(15*rice + 20*beans
           + 8*nuts + 10*greens)
s.Solve()
print(round(s.Objective().Value(), 2))
```

One line, `s.Add(nuts <= 1)`, and the cheapest lunch rises from 40.97 to 43.28 coins: about 2 coins per pupil buy a rule people need.

5 **True**

OPTIMAL means best for the rules and numbers in the model, nothing more.